perplexity

B-prompt.md

The first defect to fix is `placedBefore`: it rejects the supplied example's topological schedules and accepts dependency-reversed schedules. Independently, `Planning` quantifies over dataflows for which its laws cannot all hold, and the run record is not connected by laws to the result key. Below, numbers refer to the nine requested review categories; omitted categories have no additional verified finding. All quotations are from the supplied bundle, frozen at `2710cfc`.[1]

`demand.lth`

2. Laws that do not determine what they claim

```
RecordsEveryCutoff ... m.recorded d h == { entry := d.entry, cone := h,
parameters := d.sets, ... }
```

This determines the listed fields exactly, but does not record `d.decoration`. Two demands differing only in decoration produce the same record. Consequently, the record does not determine the demanded type, despite the comment that it records every parameter the demand carries.[1]

```
Run = { record : Record, result : String }
Runs : U := { rs : List Run | determined rs }
```

`determined` establishes consistency within a supplied list, not correctness or global determination. The singleton lists `[{record := r, result := "0"}]` and `[{record := r, result := "1"}]` each satisfy it. Neither is connected to execution, and `Demanding` contains no operation or law requiring a run store to inhabit `Runs`. Thus an incorrect result, or inconsistent results in separately supplied lists, passes the stated constraints.[1]

5. Against the specification

```
LawDemand = { law : String, cone : Hash, given : List (String * Value), accepts
: Grade }
```

`delta` is carried by `Demand` and preserved by `inner`, but is not passed through `lawDemand`. If a law uses the demand's sampling level, this interface does not supply it through the stated passing-down mechanism. The cost conversion is explicitly deferred; this is therefore an interface gap for such laws, not a demonstrated failure of an imported certification model.[1]

```
Record = { entry : String, cone : Hash, parameters : ..., accept : Grade, ... }
```

The record omits decoration and the seed, plan identity and certificate identities required for result identification by 8.S8. The comment delegates the latter fields to `plan.lth`, but no connecting structure or law makes that delegation effective. [1]

6. Implication over a type

```
admitsAssumptions d or not (e.grade == just AssumedGrade) or not (k.serves ...)
```

`NoPriceRefusesAssumed` tests refusal only when the demand rejects assumptions and the entry has assumed grade. Quantify over non-assumption-admitting demands and assumed entries, rather than drawing arbitrary entries and demands whose cases often make the law vacuous. [1]

The implication in `agreesWithRun` has the same testing weakness when arbitrary records almost never coincide. For testing determination, generate two runs sharing an identity explicitly. [1]

7. What I would change first

Connect a run's record to its complete result identity, including decoration and the key constructed in `plan.lth`. Otherwise the record's proposed determination property uses an identity already known to omit a result-changing input. [1]

9. Identifying a result

Missing is a law relating the actual execution context, recorded demand, plan, certificate snapshots and constructed `MemoKey`. It must establish that these are the inputs of this run—not merely independently well-shaped values—and that the record retains the key or enough information to reconstruct it. A separate store law should connect equal result identities to equal stored results. [1]

`demand_model.lut`

4. The model against its theory

```
lawDemand ... { law := law, cone := cone, given := d.sets, accepts := d.accepts
}
recorded ... { entry := d.entry, cone := h, ... }
```

These satisfy the two direct record-equality laws by construction. I cannot establish a violation of `NoPriceRefusesAssumed` or `LoweringLosesNothing` without the imported certification laws; those dependencies are outside this bundle's scope. [1]

5 and 8. Closed parameters disguised as defaults

```
theDemand ... { entry := entry, decoration := nothing, accepts := accepts,
delta := nothing, ... }
```

Neither `decoration` nor `delta` is an argument. They are fixed values, unlike the declared defaults for entry, grade and fuel.

- Fixing decoration prevents this root-demand constructor from requesting a decorated output.
- Fixing `delta` means a sampling level cannot be supplied through this constructor. This is a parameter-exposure defect; `nothing` is not itself a numeric cutoff.

The supplied fuel value `10000000` and grade `Sampled` are declared argument defaults, so I would not flag them merely for being literal values: the cited rules expressly permit declared defaults for the first demand.[1]

7. What I would change first

Expose decoration and confidence as arguments, with any permitted defaults explicitly declared. This makes the root constructor match the parameterized demand it claims to build.[1]

`plan.lth`

1. Laws that are wrong

```
subset (keep placed (readsOf ns n)) placed
```

This is automatically true: `keep placed` retains only names already in `placed`.

```
subset (readsOf ns n) (addNew placed rest)
```

This allows dependencies in the remaining schedule, so it does not enforce "none before what it reads." For `a` reading nothing and `b` reading `a`:

- `["a","b"]` passes.
- `["b","a"]` also passes.

Thus `LinearizesOn` admits an execution before its dependency.[1]

For a self-reading node `x`, neither previously placed names nor the remaining schedule contains `x` when it is visited. No duplicate-free schedule covering `x` passes. Yet:

```
plan : Strategy -> Dataflow A -> String -> Plan A
```

takes unrestricted dataflows, and `PlanIsTheLiveCone` plus `PlanLinearizes` requires such a schedule for every input. A one-node self-cycle therefore makes these requirements unsatisfiable together. The comment that reachability terminates on cycles does not supply a valid linearization of them.[1]

1 and 2. Purity does not establish declared dependence

```
Dataflow (A : U) = { nodes : List Node, init : A, step : String -> (String ->
A) -> A }
```

There is no law requiring `step n` to depend only on `readsOf nodes n`.

Counterexample: declare independent nodes `a` and `b`, set `init := 0`, and define `step a env := 1`, `step b env := env "a"`. Both orders linearize according to the declared graph, but `b` is `1` in `["a","b"]` and `0` in `["b","a"]`.

This is an allowed `Dataflow` that falsifies `PlanIsConfluent`. Since that law does not depend on the planner's chosen schedule, no planner can repair the violation. A locality law and an appropriate admissible-dataflow domain are missing. [1]

2. Hash laws do not identify complete keys or plans

```
KeySeesTheDemand ...
KeySeesTheCone ...
...
```

These prohibit collisions only between keys differing in one field. They do not prohibit a collision when two fields change together.

A concrete abstract countermodel is to injectively encode each field as a finite-support bit vector and combine the six vectors by XOR. Changing one field necessarily changes the result, satisfying all six laws; coordinated changes in two fields can cancel. This is a countermodel to the stated implication from these laws to complete-key identification, subject to the available representation of imported `Hash`. [1]

```
PlanHashSeesTheSchedule ...
```

A `planHash` depending only on the schedule passes this law while ignoring strategy, entry and live set. Those parts of the plan's identity are undetermined. [1]

3. Roles

```
Strategy : U := data { ForwardOnDemand : Strategy ; CallByNeed : Strategy ; }
```

There is no law distinguishing the operational roles of the two constructors. A planner that uses the same valid schedule for both, returning the requested strategy tag, satisfies the strategy-related laws on suitable dataflows. Tag preservation and result agreement do not state strict-forward execution versus demand-driven evaluation. [1]

5 and 9. Key construction is disconnected from execution

```
DemandKey = { decoration : Maybe Decoration, values : List (String * Value) }
```

This includes live hole assignments but has no general mechanism for other demand parameters the cone may read, such as an explicitly consumed confidence or cutoff. A cutoff supplied through `sets` can enter the key; a separately held demand field cannot. That falls short of 8.S8's rule for all read parameters. [1]

```
MemoKey = { demand : DemandKey, cone : Hash, mode : String, seed : Seed, plan :
Hash, certificates : List Hash }
```

No law establishes that:

- `plan` is the hash of the actual chosen plan;
- `certificates` are precisely the actual live-cone certificate snapshots;
- mode and seed are those used by execution;
- the chosen model is represented in the appropriate identity component;
- the run record and stored result use this same key.

Field-sensitivity laws do not establish these provenance relationships.[1]

6. Implication over a type

```
not (LinearizesOn ... s1 and LinearizesOn ... s2) or EquivalentOn ...
```

`Confluent` should quantify over pairs of linearizations of the specified live cone. Arbitrary string-list schedules will usually fail the premise, yielding successful tests without checking confluence.[1]

Likewise, `KeyReadsOnlyTheCone` should draw names from the complement of the live set when testing invariance under unread assignments.[1]

7. What I would change first

Correct `placedBefore`. Then restrict planning to admissible dataflows, with declared-dependence locality and an acyclic live graph. Correcting the order check alone does not make the current unrestricted theory satisfiable.[1]

`plan_model.lut`

4. The model against its theory

```
bothLinearize := LinearizesOn ... needPlan.schedule and LinearizesOn ...
forwardPlan.schedule
```

For the supplied example, both algorithms produce `["a","b","c","d"]`. At `b`, `placed = ["a"]` and `rest = ["c","d"]`; its dependency `a` is accepted. At final node `d`, however, `placed = ["c","b","a"]` and `rest = []`; both dependencies are accepted.

Thus this particular example's schedules do pass the defective predicate. It does not expose the reversal bug.[1]

The actual model violation is:

```
agree := Confluent ... needPlan.schedule ["a","c","b","d"] and EquivalentOn ...
```

This example also passes and yields `a=1`, `b=2`, `c=2`, `d=4`. It demonstrates agreement for this dataflow, not certification of `planning` over all dataflows.[1]

For a verified failing case, use the hidden-dependency dataflow above with entry `b` and declare `b.reads := ["a","c"]`, while declaring `a` and `c` independent. Let `a := env "c"`, `c := 1`, and `b := env "a"`.

- `byNeed` schedules `["a","c","b"]`, yielding `a=0`, `b=0`.
- `forward` schedules `["c","a","b"]`, yielding `a=1`, `b=1`.

The model breaks `StrategiesAgree` and `TheDefaultsAgree`. The dataflow is permitted because no locality law forbids the undeclared read of `c` by `a`. [1]

For a self-cycle `x.reads := ["x"]`, `byNeed` repeats `x` while `forward` omits it. Both fail `PlanLinearizes`. [1]

7. What I would change first

Make the model's domain match an admissible-dataflow theory rather than declaring `planning : Planning Nat eqNat` over unrestricted records. The example is sound but cannot justify the universal model claim. [1]

`08-demand.md`

5. Specification-level issue, separate from implementation defects

> "call by need … computes the same live subgraph"

This needs qualification alongside 8.S3's requirement that both branches of a value-dependent conditional are live. A demand-driven evaluator can avoid evaluating the unselected branch; the strict forward plan evaluates both statically live branches. Equality of demanded output is not equality of evaluated nodes. The chapter should distinguish static liveness, actually evaluated nodes and output agreement. This is a specification concern, not a reopening of the decided strict-forward strategy. [1]

7. What I would change first

Clarify that distinction, so strategy laws do not require evaluating the same names merely to establish equal demanded results. [1]

Not sure

- Whether imported `Hash` is finite-width. If it is, the per-field collision-prohibition laws may already be impossible for sufficiently large field domains; this bundle does not establish its representation.
- Whether the chosen model and interpreter/platform identity are fully encoded by the imported cone hash.
- Whether imported certification laws make the two certification-dependent `Demanding` clauses automatic.
- The exact additional requirements of 12.S18: its full text is not included.
- Host claims about `run.py`, `cli.py` and JSON serialization; the source files are not supplied.
- No law can currently be checked for the deferred decoration implementation or backward extraction from `Program`, since those remain holes. [1]

⁂

1. B-prompt.md